

Appendix

A Proofs

LEMMA 1. Consider an input query Q with two endpoints at index-locations s and e , and a set of fair range queries $\{x, y_1\}, \{x, y_2\}, \dots$ with the index-location of the starting endpoint as x . The most similar fair range query is one among two ranges — has the endpoint of y_i (i) where $y_i \leq e$ and among all such ranges $e - y_i$ is the smallest, (ii) where $y_i \geq e$ and among all such ranges $y_i - e$ is the smallest.

PROOF. For this proof, we use the similarity function of the Tversky index with input parameters $\alpha \geq 0$ and $\beta \geq 0$. Consider the scenario where x lies on the left of s . Any fair range that has some intersection with Q , will add a cost of $\beta(s - x)$ for the denominator with no addition to the numerator. Let us now consider all index-locations y_i which lie inside the range, i.e., $y_i \leq e$. In such a case, the intersection would be $y_i - s$ which forms our numerator and is a term present in the denominator and thus needs maximizing. The part that is left out, $e - y_i$ is considered as a penalty with a weight of α and needs to be minimized. Hence, among all values of y_i which satisfy $y_i \leq e$, we choose the one that is maximum, i.e., nearest to e .

For the case when $y_i \geq e$, the numerator already consists of $e - s - 1$ as the whole input range intersects with the range $[x, y_i]$. The same argument for the penalty term for the part $s - x$ holds. As y_i grows farther away from e an additional error of $\beta(y_i - e)$ is incurred, and hence, we choose the minimum value of y_i for the case $y_i \geq e$.

A similar argument extends to the case when x is located inside the range. We skip this proof due to a lack of space. $\square$

THEOREM 1. Given a database $\mathcal{D}$, fairness criteria $\mathcal{F}$, UFRQ 1 produces a output range Q' that is fair and among all fair ranges is the most similar fair range to the input range.

PROOF. We prove both feasibility and optimality.

Feasibility. For every starting index x , Algorithm 1 constructs the range-search query directly from the fairness criteria $\mathcal{F}$ using the transformations developed in Sections 3.2, 3.3, and 3.4. By construction,

- Every ADDP constraint is transformed into an interval constraint over one dimension of $\mathcal{D}_T$ (Equation 13),
- Every RDP constraint is transformed into two range constraints (Equations 15 and 16), and
- Coverage constraints are enforced separately through the CSL structure together with the verification step.

Consequently, every endpoint returned by the Range Tree satisfies all transformed fairness constraints, which are equivalent to the original fairness criteria. The additional coverage verification guarantees that the returned range also satisfies all coverage constraints. Therefore every candidate considered by the algorithm is fair.

Optimality for a fixed starting index. Fix a starting index x . The Range Tree returns all feasible endpoints satisfying the transformed fairness constraints. By Lemma 1, among all feasible endpoints it is sufficient to consider only the predecessor and successor of the target endpoint (either e or the smallest coverage-satisfying endpoint obtained from CSL). These are exactly the candidates

returned by the Range Query on our Tree data structure. Hence, the algorithm selects the fair range having the maximum similarity among all fair ranges whose left endpoint is fixed at x .

Global optimality. The algorithm repeats the above procedure for every possible starting index $x \in \{0, \dots, e\}$. Every fair range whose similarity with the input query can be non-zero must have its left endpoint in this interval, and therefore every feasible fair range is considered in exactly one iteration. Since the optimal fair range for each fixed x is computed, taking the maximum similarity over all iterations yields the most similar fair range globally.

Hence, Algorithm 1 always returns a fair range and, among all such fair ranges, returns the one having the maximum similarity to the input query. $\square$

THEOREM 3. FRQP with a single ADDP (or RDP or ratio-based coverage) constraint is NP-complete when the number of dimensions d is not fixed.

PROOF. We first go over the verification version of the problem and then show the reduction.

Verification: Consider the decision version of the problem, where we are asked to find a range that has at least τ similarity with the input range while maintaining a fairness threshold of ϵ for the ADDP (δ for RDP or θ for ratio-based coverage). Given a signature (an output range), we can compute the overlap with the input range in linear time and, thus, compute the similarity. In the same linear scan, we can also compute if the query is fair by computing the count of the relevant sensitive groups. Thus, verification can be completed in linear time.

Reduction: For this reduction, we consider an instance of MBP. Given a set of points $\mathcal{P} = P^+ \cup P^-$, find an axis aligned hyper-rectangle with maximum cardinality such that it only contains points from P^+ . For the decision version of FRQP, we create a $d + 1$ dimensional database $\mathcal{D}$ where the first d dimensions are numerical in nature, and the last dimension is the sensitive attribute. For every point $p \in P^-$, we create a tuple $t \in \mathcal{D}$ where the sensitive attribute for t is $-$ and the numerical attribute of t is the same as p . For every point $p \in P^+$, we create two tuples t^+ and t^- , both of which have the same value for the numerical attributes as p . The tuple t^+ has the sensitive attribute set to $+$, and the tuple t^- has the sensitive attribute set to $-$. Additionally, for our decision problem, we set the weight of 1 for the sensitive group $+$ and 1 for $-$. For the ADDP criterion we set $\epsilon = 0$ as below:

$$\text{abs}(|O(Q, +)| - |O(Q, -)|) = 0$$

For the RDP criterion, we set $\delta = 0$ as below:

$$\frac{|O(Q, +)|}{|O(Q, -)|} = 1$$

For the ratio-based coverage constraints we set $\theta = 1/2$:

$$\frac{|O(Q, +)|}{|O(Q, -)|} \geq \frac{1}{2}$$

For the input range, we set the hyper-rectangle that contains all the points in $\mathcal{D}$.

The transformation is polynomial as the creation of points took $O(n(1 + d))$ time. Creation of the input rectangle took linear time to find the boundaries along each direction. Hence, the reduction

Table 9: Table of notations

Notations	Description
$\mathcal{D}$	Database which gives as part of the FRQP
$\mathcal{A}$	Set of d numerical attributes (columns used for range filtering).
n	Size of the database $\mathcal{D}$
Q	Range query specified as part of FRQP
ϵ	Threshold for ADDP-criteria (see Equation 2)
δ	Threshold parameter for RDP-criteria (see Equation 3)
θ	Threshold for Coverage criteria (see Equation 4)
α	Parameter that scales the term $ A - B $ in Tversky index ($\mathcal{S}_T(A, B)$)
β	Parameter that scales the term $ B - A $ in $\mathcal{S}_T(A, B)$
$\mathcal{F}$	Set of fairness criteria specified by the user
C	Cumulative Sum Array (CSA) used for efficient prefix counting of groups.
CSL	Coverage Search Lookup array generated by the two-pointer scan.
$\mathcal{D}_T$	Transformed database containing computed disparity dimensions.
M	A sufficiently large positive constant utilized in the MILP formulation.
L_j (U_j respectively)	Lower (Upper respectively) boundary location indices along attribute A_j .
λ_i	Value of Tversky similarity optimized in MIFLP (Dinkelbach's algorithm) in iteration i .
w_i, w_j	Pre-computed weight (population scaling factors) assigned to groups g_i and g_j respectively.
$O(Q)$	Set of tuples returned by query Q .
$O(Q, g_i)$	Multi-set of tuples in the output of query Q that belong to sensitive group g_i .
$g_i, g_j, \dots$	Specific sensitive group or conjunction of group.
$\mathcal{S}$	Set of categorical sensitive attributes in the database $\mathcal{D}$

has taken polynomial time. We now reason about the correctness of our reduction.

Observe that every point $p \in P^+$ is now transformed to FRQP as two points which are co-located, but with opposite sensitive groups. Hence, including one point in any range also includes the other point. Now, consider a valid solution to MBP. A hyper-rectangle which contains the maximum number of points from P^+ maps to a fair range as the ADDP disparity for such a range is 0, RDP ratio is 1 and ratio based coverage score of $1/2$. Additionally, note that any similarity measure (Tversky with non-negative α and β) maps to maximizing the cardinality as the union is already equal to all the points in the database $\mathcal{D}$, i.e., $2|P^+| + |P^-|$. Hence, a solution to MBP which has a cardinality of size at least 2γ also shows up as a fair range which has similarity of at least $(2\gamma)/(2|P^+| + |P^-|)$.

For the reverse direction, consider a fair range to FRQP which has similarity of at least $(2\gamma)/(2|P^+| + |P^-|)$. As a fair range in FRQP ensures by definition that it contains a subset of points from P^+ . By definition, no point from P^- can be present in the hyper-rectangle for MBP. As the similarity is at least $(2\gamma)/(2|P^+| + |P^-|)$, and each co-located point maps back to one point in the MBP space, we have at least γ points in the output range. Hence, proved. $\square$

B Algorithms

C Future work and conclusion

We study the problem of Fair Range Query Problem — FRQP and propose a generalized framework that brings together various forms of Similarity measures like Jaccard, Dice-Sørensen index, and Tversky index and various fairness measures like Demographic parity (ADDP and RDP) and Coverage criteria. We developed a spatial data-structures based approach for the single predicate range queries. Empirically, we show that among the various spatial data-structures k - d tree outperforms other spatial data-structures. We also prove

Algorithm 2: PreProcessing — Build the spatial tree data structure

Input: Dataset $\mathcal{D}$, query Q , fairness criteria $\mathcal{F}$

Output: Transformed dataset $\mathcal{D}_T$, CSA C , CSL

```

1 Sort  $\mathcal{D}$  on range attribute  $A_i$ 
2 Construct cumulative sum array  $C$ 
3 if  $\mathcal{F}$  contains coverage constraints then
4   | Construct CSL using two-pointer scan (ConstructCSL)
5  $\mathcal{D}_T \leftarrow \emptyset$ 
6 for  $x \leftarrow 0$  to  $n - 1$  do
7   |  $t'_x \leftarrow \emptyset$ 
8   | foreach ADDP criterion  $(g_i, g_j, w_i, w_j, \epsilon)$  do
9   |   | Append  $w_i C[x][g_i] - w_j C[x][g_j]$  to  $t'_x$ 
10  | foreach RDP criterion  $(g_i, g_j, w_i, w_j, \delta)$  do
11  |   | Append  $w_j C[x][g_j] - (1 + \delta) w_i C[x][g_i]$  to  $t'_x$ 
12  |   | Append  $(1 + \delta) w_j C[x][g_j] - w_i C[x][g_i]$  to  $t'_x$ 
13  |   | append index location  $x$  to  $t'_x$ 
14  |   | Insert  $t'_x$  into  $\mathcal{D}_T$ 
15 Build Range Tree  $R_T$  on  $\mathcal{D}_T$ 
16 return  $(R_T, C, CSL)$ 
```

that given $\tilde{O}(n)$ space, Range-tree provides a $\tilde{O}(n)$ time approach to find the optimum query Q' . For multi-predicate queries where number of dimensions can vary as part of the input we show the hardness results even when there is one ADDP or RDP criteria. We develop a Mixed Integer Fractional Linear Programming formulation that finds the most similar fair range. Empirical evaluation shows that our methods outperforms the baseline Best First Search.

Algorithm 3: ConstructCSL**Input:** Sorted dataset $\mathcal{D}$, coverage constraints $\mathcal{F}_c$ **Output:** CSL

```

1  $s_c \leftarrow 0; \quad e_c \leftarrow 0$ 
2 while  $s_c < n$  do
3   while  $e_c < n$  do
4     if  $[s_c, e_c]$  fails coverage constraints then
5        $e_c \leftarrow e_c + 1$ 
6   if  $e_c < n$  then
7      $\text{CSL}[s_c] \leftarrow e_c$ 
8    $s_c \leftarrow s_c + 1$ 
9 return CSL

```

An important property that all our algorithms are designed with is the *Anytime* property. Our approach starts with a solution to the problem and given more time, it constantly improves the solution until finding the optimum.

Future Work: An interesting open problem, is to extend our framework to multi-table joins. Another interesting direction is to look at an interactive framework where the user/analyst can modify the query and parameters interactively. On the similarity measures front, one can consider defining similarity measures based on query boundaries — Jaccard similarity based on the area/volume of the query regions (input query region defined by Q and the output by Q'). In this case, the single-predicate technique extends trivially to this setting. However, extending the multi-predicate framework seems to be non-trivial. We plan to work on these open problems as part of future work.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009